PROJECT REPORT

SECURE FILE TRANSFER MONITORING SYSTEM

TITLE PAGE

SECURE FILE TRANSFER MONITORING SYSTEM

A Project Report
Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE / INFORMATION TECHNOLOGY

Submitted By

AMANDEEP SINGH

Under the Guidance of

[YOGANANDA COLLEGE OF ENGINEERING AND TECHNOLOGY]
[JAMMU UNIVERSITY]

2026

CERTIFICATE

This is to certify that the project report entitled "Secure File Transfer Monitoring System" has been successfully completed by Amandeep Singh under my guidance and supervision.

The project has been developed as part of the academic requirements for the award of the degree of Bachelor of Engineering.

The work presented in this report is based on the implementation, testing, and analysis carried out during the project.

Project Guide: _____

Head of Department: _____

Date: _____

Place: _____

DECLARATION

I hereby declare that the project entitled "Secure File Transfer Monitoring System" is my original work carried out for academic and educational purposes.

The project was developed using Python, the Watchdog filesystem monitoring library, SHA-256 hashing, JSON-based hash storage, and Python logging.

The information and results presented in this report are based on the implementation and testing performed during the development of the project.

Name: Amandeep Singh

Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, faculty members, and institution for providing me with the opportunity and support to complete this project titled "Secure File Transfer Monitoring System."

I am thankful to my project guide for providing valuable guidance, suggestions, and technical support throughout the development of this project.

I would also like to thank my friends and colleagues for their encouragement and assistance during the implementation and testing phases.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this project.

Amandeep Singh

ABSTRACT

The Secure File Transfer Monitoring System is a Python-based cybersecurity project designed to monitor file activities within a selected directory and identify potential unauthorized

modifications and file movements.

File operations such as creation, modification, deletion, and movement are common in computer systems. However, unauthorized changes to important files can result in data leakage, data loss, file tampering, malware-related activity, or other security incidents. Therefore, monitoring file activity is an important part of cybersecurity and security auditing.

The proposed system uses the Python Watchdog library to continuously monitor a specified directory. Whenever a file is created, modified, deleted, or moved, the system detects the corresponding filesystem event and records it.

The system also implements SHA-256 hashing for file integrity verification. When a file is created, its SHA-256 hash is calculated and stored in a JSON file. When the file is subsequently modified, a new hash is calculated and compared with the previously stored hash. If the two hashes are different, the system generates an Integrity Violation alert.

Security events are recorded in a log file using Python's logging module. These logs provide an audit trail that can be used for reviewing and investigating file activity.

The project was implemented and tested in a Kali Linux environment using Python 3.13 and Watchdog 6.0.0. The project demonstrates practical concepts of File Integrity Monitoring, filesystem monitoring, security logging, event detection, and basic cybersecurity monitoring.

TABLE OF CONTENTS

- Future Scope
- Conclusion
- References
- Appendix

CHAPTER 1: INTRODUCTION

1.1 Introduction

Modern organizations depend heavily on digital files for storing and exchanging important information. Files may contain personal information, business documents, credentials, configuration information, financial information, and other sensitive data.

During normal computer usage, files are frequently created, copied, modified, moved, and deleted. However, unauthorized file operations can create serious security risks.

For example, an employee may accidentally move confidential information to an inappropriate location, an attacker may modify an important configuration file, or malicious software may replace or modify existing files.

A security monitoring system can help identify such activities by continuously observing file operations and maintaining an audit trail.

The Secure File Transfer Monitoring System is designed as a simple and lightweight solution for monitoring file activities in a specified directory.

1.2 Project Background

File Integrity Monitoring (FIM) is a security technique used to identify changes to files and directories.

One common method of verifying file integrity is the use of cryptographic hashes.

A cryptographic hash converts file content into a fixed-length value.

For example:

File
↓
SHA-256 Algorithm
↓
Hash Value

If the contents of the file change, the resulting hash will normally change.

Therefore:

Original File
↓
Original Hash

File Modified
↓
New Hash

Original Hash ≠ New Hash
↓
Possible Integrity Violation

This concept is used by the project to detect unexpected file modifications.

CHAPTER 2: PROBLEM STATEMENT

2.1 Problem Statement

Files containing important or confidential information may be created, modified, moved, or deleted without proper authorization.

Without a dedicated monitoring mechanism, it can be difficult to identify when these activities occur and maintain an audit trail of the events.

The project therefore aims to develop a simple Python-based monitoring system capable of:

Monitoring a selected directory.
Detecting file creation.
Detecting file modification.
Detecting file deletion.
Detecting file movement.
Calculating SHA-256 hashes.
Detecting file integrity violations.
Maintaining security logs.

CHAPTER 3: PROJECT OBJECTIVES

The main objectives of the project are:

I. To develop a simple file monitoring system using Python.
II. To monitor a selected directory continuously.
III. To detect file creation events.
IV. To detect file modification events.
V. To detect file deletion events.
VI. To detect file movement events.
VII. To calculate SHA-256 hashes for monitored files.
VIII. To compare previous and current hash values.
IX. To generate alerts when file integrity is violated.

X. To maintain detailed security logs.
XI. To provide a foundation for future cybersecurity and SIEM integration.

CHAPTER 4: SCOPE OF THE PROJECT

The current project focuses on filesystem monitoring within a selected directory.

The system provides the following functionality:

File Creation Monitoring

Detects when a new file is created inside the monitored directory.

File Modification Monitoring

Detects when an existing file is modified and compares its SHA-256 hash.

File Deletion Monitoring

Detects when a monitored file is deleted.

File Movement Monitoring

Detects when a file is moved from one location to another.

File Integrity Verification

Uses SHA-256 to identify changes in file content.

Security Logging

Stores security events in a log file.

CHAPTER 5: EXISTING SYSTEM

In a basic computer environment, users can create, modify, move, and delete files without a dedicated security monitoring application.

Operating systems provide various file management and auditing features, but configuring and analyzing these features may be complex for beginners.

A basic file management environment may not provide a simple centralized view of:

File creation
File modification
File deletion

File movement
File integrity

Therefore, a lightweight monitoring solution can be useful for educational and basic security monitoring purposes.

CHAPTER 6: PROPOSED SYSTEM

The proposed Secure File Transfer Monitoring System provides a simple Python-based solution for monitoring filesystem activity.

The system uses the Watchdog library to monitor a selected directory.

The basic workflow is:

Monitored Folder
↓
Watchdog Observer
↓
Filesystem Event
↓
Create / Modify / Delete / Move
↓
Process Event
↓
SHA-256 Integrity Check
↓
Security Log
↓
Alert if Required

The proposed system is lightweight, easy to understand, and suitable for learning basic cybersecurity monitoring concepts.

CHAPTER 7: REQUIREMENTS ANALYSIS

7.1 Hardware Requirements

The project does not require specialized hardware.

Component	Requirement
Processor	Dual-Core or above
RAM	4 GB minimum
Storage	At least 10 GB free space
Keyboard	Required
Mouse	Required
Network	Optional

7.2 Software Requirements

Software	Version/Description
Operating System	Kali Linux
Python	3.13

Watchdog 6.0.0
Shell Zsh
Text Editor Nano
Virtual Environment Python venv

7.3 Python Libraries

The project uses:

os
json
hashlib
logging
watchdog
os

Used for creating and managing project directories.

json

Used for storing file hashes in JSON format.

hashlib

Used to calculate SHA-256 hashes.

logging

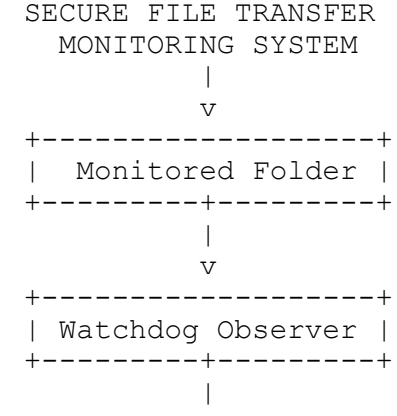
Used to create security audit logs.

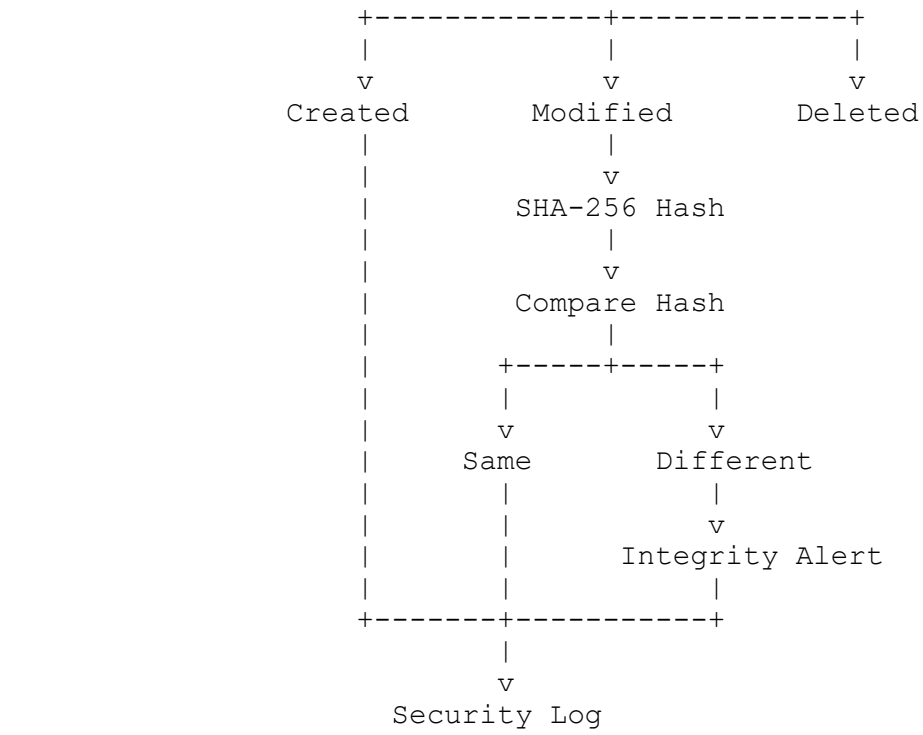
watchdog

Used to monitor filesystem events.

CHAPTER 8: SYSTEM ARCHITECTURE

The architecture of the project is:





CHAPTER 9: METHODOLOGY

9.1 Directory Monitoring

The project first defines a directory that needs to be monitored.

monitored_folder/

The Watchdog observer continuously watches this directory.

9.2 File Event Detection

The system detects four major filesystem events:

CREATE
MODIFY
DELETE
MOVE

Each event is handled by a corresponding function.

on_created()
on_modified()
on_deleted()
on_moved()
9.3 Hash Calculation

When a file needs to be verified, the system calculates its SHA-256 hash.

The process is:

File
↓
Read File Content
↓
SHA-256 Algorithm
↓
Hash Value

The hash is stored for future comparison.

9.4 Hash Comparison

When a file is modified, the system calculates a new hash.

The old hash and new hash are compared.

Old Hash
|
| Compare
|
New Hash

If:

Old Hash = New Hash

the file content has not changed.

If:

Old Hash ≠ New Hash

the system reports:

INTEGRITY VIOLATION

9.5 Security Logging

The system stores security events in:

logs/security.log

Example:

```
INFO | FILE CREATED | monitored_folder/important.txt
WARNING | FILE DELETED | monitored_folder/test.txt
WARNING | FILE MOVED | FROM=... | TO=...
WARNING | INTEGRITY VIOLATION | important.txt
```

CHAPTER 10: TECHNOLOGIES USED

10.1 Python

Python is used as the primary programming language because it provides simple syntax and many libraries suitable for cybersecurity and automation.

10.2 Watchdog

Watchdog is a Python library that provides filesystem event monitoring.

It allows the application to detect events such as:

File creation
File modification
File deletion
File movement
10.3 SHA-256

SHA-256 is a cryptographic hashing algorithm.

It is used in this project as a mechanism for checking file integrity.

The hash acts like a digital fingerprint of the file contents.

10.4 JSON

JSON is used to store file paths and their corresponding SHA-256 hashes.

Example:

```
{
    "monitored_folder/important.txt": "HASH_VALUE"
}
```

10.5 Python Logging

Python's logging module is used to create an audit trail.

Security events are stored in:

logs/security.log

CHAPTER 11: SYSTEM IMPLEMENTATION

11.1 Project Directory

The project was created using the following structure:

```
Secure-File-Transfer-Monitor/
|
├── data/
|   └── hashes.json
|
```

```
├── logs/
│   └── security.log
│
├── monitored_folder/
│
├── screenshots/
│
├── monitor.py
├── requirements.txt
└── venv/
```

11.2 Creating the Project

The project directory was created using:

```
mkdir Secure-File-Transfer-Monitor
cd Secure-File-Transfer-Monitor
```

The required directories were created using:

```
mkdir monitored_folder
mkdir logs
mkdir data
mkdir screenshots
```

11.3 Python Virtual Environment

A virtual environment was created using:

```
python3 -m venv venv
```

The environment was activated using:

```
source venv/bin/activate
```

The project was tested using Python 3.13.

11.4 Installing Watchdog

The Watchdog library was installed using:

```
pip install watchdog
```

The installed version during testing was:

```
watchdog==6.0.0
```

CHAPTER 12: CODE IMPLEMENTATION

12.1 Importing Libraries

The program starts by importing the required libraries:

```
import os
```

```
import json
import hashlib
import logging

from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
```

These libraries provide directory management, JSON storage, hashing, logging, and filesystem monitoring functionality.

12.2 Defining Project Paths

```
MONITORED_FOLDER = "monitored_folder"
HASH_FILE = "data/hashes.json"
LOG_FILE = "logs/security.log"
```

These variables define:

The folder to monitor.
The location of the hash database.
The location of the security log.
12.3 Creating Required Directories

```
os.makedirs("data", exist_ok=True)
os.makedirs("logs", exist_ok=True)
```

These commands ensure that the required directories exist before the program starts.

12.4 SHA-256 Hash Function

The main hash function is:

```
def calculate_hash(file_path):

    sha256 = hashlib.sha256()

    try:
        with open(file_path, "rb") as file:

            while chunk := file.read(4096):
                sha256.update(chunk)

        return sha256.hexdigest()

    except (FileNotFoundError, PermissionError):
        return None
```

This function reads the file and generates its SHA-256 hash.

12.5 Loading Hashes

```
def load_hashes():

    try:
```

```
        with open(HASH_FILE, "r") as file:
            return json.load(file)

    except FileNotFoundError:
        return {}
```

This function loads previously stored hashes.

12.6 Saving Hashes

```
def save_hashes(hashes):

    with open(HASH_FILE, "w") as file:
        json.dump(hashes, file, indent=4)
```

This function saves the hash information to the JSON file.

12.7 File Monitoring Class

The monitoring class is created using:

```
class FileMonitor(FileSystemEventHandler):
```

The class contains handlers for different file events.

12.8 File Creation

The on_created() function detects newly created files.

The system:

Gets the file path.
Calculates the SHA-256 hash.
Stores the hash.
Records the event.
Displays a message.

Example output:

```
[+] FILE CREATED: monitored_folder/important.txt
```

12.9 File Modification

The on_modified() function calculates the new hash and compares it with the stored hash.

The main detection logic is:

```
if old_hash and new_hash != old_hash:
```

If the hashes are different, the system generates:

```
[!] INTEGRITY VIOLATION
```

12.10 File Deletion

The on_deleted() function detects file deletion.

Example output:

[!] FILE DELETED: monitored_folder/test.txt

The deletion is also written to the security log.

12.11 File Movement

The on_moved() function records the source and destination paths.

Example:

[!] FILE MOVED:
monitored_folder/test.txt ->
monitored_folder/backup/test.txt
CHAPTER 13: RUNNING THE SYSTEM

The monitoring system is started using:

python3 monitor.py

The expected output is:

```
==================================================
 Secure File Transfer Monitoring System
==================================================
Monitoring folder: monitored_folder
Press CTRL+C to stop.
```

The program continues running until the user presses:

CTRL+C

CHAPTER 14: TESTING AND RESULTS

The system was tested using multiple file operations.

14.1 Test Case 1 — File Creation

Command:

```
echo "Confidential Information" > monitored_folder/important.txt
```

Expected output:

[+] FILE CREATED: monitored_folder/important.txt
Result

PASS

The system successfully detected the creation of the file.

14.2 Test Case 2 — Hash Storage

Command:

```
cat data/hashes.json
```

The system displays the stored file hash.

Example:

```
{
    "monitored_folder/important.txt": "SHA256_HASH_VALUE"
}
```

Result

PASS

The SHA-256 hash was successfully stored.

14.3 Test Case 3 — File Modification

Command:

```
echo "Unauthorized modification" >>
monitored_folder/important.txt
```

Expected output:

```
[!] INTEGRITY VIOLATION: monitored_folder/important.txt
```

Result

PASS

The system detected a difference between the old and new hash.

14.4 Test Case 4 — File Movement

Command:

```
mkdir monitored_folder/backup
mv monitored_folder/test.txt monitored_folder/backup/
```

Expected output:

```
[!] FILE MOVED:
monitored_folder/test.txt ->
monitored_folder/backup/test.txt
```

Result

PASS

The system successfully detected the file movement.

14.5 Test Case 5 — File Deletion

Command:

rm monitored_folder/backup/test.txt

Expected output:

[!] FILE DELETED: monitored_folder/backup/test.txt
Result

PASS

The system successfully detected the deletion.

CHAPTER 15: TESTING SUMMARY

Test Case	Activity	Expected Result	Result
TC-01	File creation	Creation detected	PASS
TC-02	Hash generation	Hash stored	PASS
TC-03	File modification	Integrity alert	PASS
TC-04	File movement	Movement detected	PASS
TC-05	File deletion	Deletion detected	PASS
TC-06	Security logging	Event recorded	PASS

The testing demonstrated that the core functionality of the system worked successfully in the test environment.

CHAPTER 16: SCREENSHOTS AND PROJECT EVIDENCE

The following screenshots should be included in the final report.

Figure 1: Project Directory Creation

Insert screenshot showing:

mkdir Secure-File-Transfer-Monitor
Figure 2: Installing Watchdog

Insert screenshot showing:

pip install watchdog

and:

Successfully installed watchdog-6.0.0
Figure 3: Project Structure

Insert screenshot showing:

data
logs
monitored_folder
screenshots
monitor.py
requirements.txt
Figure 4: Monitoring System Running

Insert screenshot showing:

Secure File Transfer Monitoring System
Monitoring folder: monitored_folder
Press CTRL+C to stop.
Figure 5: File Creation Detection

Insert screenshot showing:

[+] FILE CREATED
Figure 6: SHA-256 Hash Database

Insert screenshot showing:

cat data/hashes.json
Figure 7: Integrity Violation

Insert screenshot showing:

[!] INTEGRITY VIOLATION

This is one of the most important screenshots in the report.

Figure 8: File Movement Detection

Insert screenshot showing:

[!] FILE MOVED
Figure 9: File Deletion Detection

Insert screenshot showing:

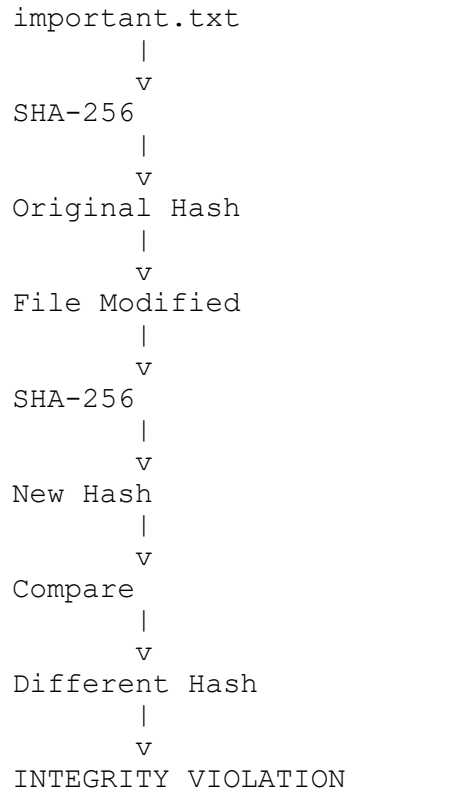
[!] FILE DELETED
Figure 10: Security Log

Insert screenshot showing:

cat logs/security.log
CHAPTER 17: SECURITY ANALYSIS

The project demonstrates a basic File Integrity Monitoring (FIM) approach.

Consider the following scenario:



This approach can help identify unexpected changes to important files.

However, a hash mismatch alone does not prove that an attack occurred. It only indicates that the file content changed. Additional information such as the user, process, source, destination, and surrounding system activity would be required for deeper investigation.

CHAPTER 18: ADVANTAGES

The main advantages of the project are:

Simple implementation.
Lightweight monitoring system.
Real-time filesystem event detection.
SHA-256 integrity verification.
Security event logging.
Easy to install and configure.
Uses open-source technologies.
Suitable for cybersecurity education.
Easy to extend with additional security features.
Provides a foundation for SIEM integration.
CHAPTER 19: LIMITATIONS

The current version has several limitations.

19.1 User Identification

The current system does not reliably identify which user performed each filesystem operation.

19.2 Process Identification

The current version does not identify the exact process responsible for an event.

19.3 Network Monitoring

The system primarily monitors filesystem activity and does not directly inspect network traffic.

19.4 USB Monitoring

The current version does not directly identify USB file transfers.

19.5 Cloud Monitoring

Cloud synchronization applications are not specifically identified.

19.6 Alerting

Alerts are currently displayed in the terminal and written to a log file.

19.7 Hash Database

The hash database is stored in a simple JSON file rather than a dedicated database.

CHAPTER 20: FUTURE SCOPE

The project can be improved significantly in future versions.

20.1 User Tracking

The system can record the operating-system user associated with file activity.

20.2 Process Tracking

The system can use process-monitoring functionality to identify the application responsible for file operations.

20.3 USB Monitoring

The project can be extended to detect removable storage devices and monitor file activity involving them.

20.4 Network Share Monitoring

The system can monitor directories mounted from network shares.

20.5 Sensitive File Detection

Rules can be added for sensitive files such as:

*.pdf
*.docx
*.xlsx
*.pem
*.key
*.env

20.6 Email Alerts

The system could send an email when a critical integrity violation occurs.

20.7 Web Dashboard

A web interface could display:

Total Events
File Creations
File Modifications
File Deletions
File Movements
Integrity Violations
20.8 SIEM Integration

Future versions could send logs to security monitoring platforms such as:

Wazuh
Splunk
Elasticsearch

This would allow the project to become part of a broader Security Operations Center monitoring environment.

CHAPTER 21: CONCLUSION

The Secure File Transfer Monitoring System was successfully developed as a lightweight Python-based cybersecurity monitoring project.

The system continuously monitors a selected directory and detects

important filesystem events including file creation, modification, deletion, and movement.

SHA-256 hashing was implemented to provide file integrity verification. When a monitored file was modified, the system calculated a new hash and compared it with the previously stored hash. A difference between the hashes resulted in an Integrity Violation alert.

The project also maintains security audit logs that provide useful information about detected file activity.

Through this project, practical knowledge was gained in Python programming, filesystem monitoring, cryptographic hashing, security logging, event detection, and File Integrity Monitoring.

Although the current system is a basic educational implementation, it provides a strong foundation for future improvements such as user tracking, process identification, USB monitoring, dashboard development, automated notifications, and integration with SIEM platforms such as Wazuh and Splunk.

Overall, the project demonstrates how a simple monitoring system can be used as a starting point for detecting suspicious file activity and supporting cybersecurity investigations.

REFERENCES

Python Documentation — Python Programming Language Documentation.
Python hashlib Documentation — Cryptographic hashing functionality.
Python logging Documentation — Application and security logging.
Watchdog Documentation — Python filesystem monitoring library.
OWASP — Cybersecurity and application security resources.
MITRE ATT&CK — Enterprise cybersecurity techniques and detection concepts.

APPENDIX A: COMPLETE SOURCE CODE

The complete source code of monitor.py is included below.

```
import os
import json
import hashlib
import logging

from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler


MONITORED_FOLDER = "monitored_folder"
HASH_FILE = "data/hashes.json"
LOG_FILE = "logs/security.log"
```

```
os.makedirs("data", exist_ok=True)
os.makedirs("logs", exist_ok=True)


logging.basicConfig(
    filename=LOG_FILE,
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)


def calculate_hash(file_path):

    sha256 = hashlib.sha256()

    try:
        with open(file_path, "rb") as file:

            while chunk := file.read(4096):
                sha256.update(chunk)

        return sha256.hexdigest()

    except (FileNotFoundError, PermissionError):
        return None


def load_hashes():

    try:
        with open(HASH_FILE, "r") as file:
            return json.load(file)

    except FileNotFoundError:
        return {}


def save_hashes(hashes):

    with open(HASH_FILE, "w") as file:
        json.dump(hashes, file, indent=4)


class FileMonitor(FileSystemEventHandler):

    def on_created(self, event):

        if event.is_directory:
            return

        file_path = event.src_path
```

```
        file_hash = calculate_hash(file_path)

        hashes = load_hashes()

        if file_hash:
            hashes[file_path] = file_hash
            save_hashes(hashes)

        logging.info(
            f"FILE CREATED | {file_path} | SHA256={file_hash}"
        )

        print(f"[+] FILE CREATED: {file_path}")


    def on_modified(self, event):

        if event.is_directory:
            return

        file_path = event.src_path

        new_hash = calculate_hash(file_path)

        hashes = load_hashes()

        old_hash = hashes.get(file_path)

        if old_hash and new_hash != old_hash:

            logging.warning(
                f"INTEGRITY VIOLATION | {file_path} | "
                f"OLD_HASH={old_hash} | NEW_HASH={new_hash}"
            )

            print(f"[!] INTEGRITY VIOLATION: {file_path}")

        elif new_hash:

            logging.info(
                f"FILE MODIFIED | {file_path} | SHA256
={new_hash}"
            )

        if new_hash:
            hashes[file_path] = new_hash
            save_hashes(hashes)


    def on_deleted(self, event):
```

```
        if event.is_directory:
            return

        file_path = event.src_path

        hashes = load_hashes()

        hashes.pop(file_path, None)

        save_hashes(hashes)

        logging.warning(
            f"FILE DELETED | {file_path}"
        )

        print(f"[!] FILE DELETED: {file_path}")


    def on_moved(self, event):

        if event.is_directory:
            return

        logging.warning(
            f"FILE MOVED | FROM={event.src_path} | "
            f"TO={event.dest_path}"
        )

        print(
            f"[!] FILE MOVED: "
            f"{event.src_path} -> {event.dest_path}"
        )


if __name__ == "__main__":

    print("=" * 50)
    print(" Secure File Transfer Monitoring System")
    print("=" * 50)

    print(f"Monitoring folder: {MONITORED_FOLDER}")
    print("Press CTRL+C to stop.")
    print()

    event_handler = FileMonitor()

    observer = Observer()

    observer.schedule(
        event_handler,
        MONITORED_FOLDER,
        recursive=True
```

```
    )

    observer.start()

    try:

        while True:
            pass

    except KeyboardInterrupt:

        observer.stop()

        print("\n[+] Monitoring stopped.")

    observer.join()
```

APPENDIX B: REQUIREMENTS

```
The project requires the following Python package:

watchdog

Installation:

pip install -r requirements.txt

The current tested version was:

watchdog==6.0.0
```

APPENDIX C: SAMPLE TEST COMMANDS

```
Start the monitor
cd ~/Secure-File-Transfer-Monitor
source venv/bin/activate
python3 monitor.py
Create a file
echo "Confidential Information" > monitored_folder/important.txt
Modify a file
echo "Unauthorized modification" >>
monitored_folder/important.txt
Create backup directory
mkdir monitored_folder/backup
Move a file
mv monitored_folder/test.txt monitored_folder/backup/
Delete a file
rm monitored_folder/backup/test.txt
View stored hashes
cat data/hashes.json
View security logs
cat logs/security.log
FINAL PROJECT SUMMARY
```

PROJECT NAME:

```
Secure File Transfer Monitoring System
```

LANGUAGE:
Python 3.13

MAIN LIBRARY:
Watchdog 6.0.0

SECURITY TECHNIQUE:
SHA-256 File Integrity Monitoring

MONITORED EVENTS:
Create
Modify
Delete
Move

HASH STORAGE:
data/hashes.json

SECURITY LOG:
logs/security.log

OPERATING SYSTEM:
Kali Linux

MAIN SECURITY ALERT:
INTEGRITY VIOLATION

End of Report